



Advanced Teleop

Control your ROS robot from your phone!



In the [previous post](#) we saw how to connect a gamepad to our system to simplify the process of driving the robot manually, called *teleoperation*. In this post we'll look at some more advanced tools, tips, and tricks to improve our teleoperation experience.

- [Streaming to a phone/tablet](#)
 - [Set up web server](#)
 - [Set up image feed](#)
 - [Set up controls](#)
 - [Run on phone/tablet](#)
- [Foxglove](#)
- [Extra Tips](#)
 - [Teleop from RViz](#)
 - [Stamped/Unstamped Twist](#)
 - [Gazebo/Sim Time](#)
 - [Launch files](#)
 - [Using the old Linux Joystick Interface](#)
 - [New teleop_tools package](#)
- [Conclusion](#)

Streaming to a phone/tablet

The arrangement we had at the end of the previous tutorial is pretty good. We can sit back at our workstation, with our gamepad and RViz, driving our robot around, seeing what it's seeing.

But sometimes we want to be more mobile than that. A great example of that is with the Spot dogs from Boston Dynamics. In their demos we see operators and robots working together in places like construction sites, and the operator wanders around with this handheld device from which they can control everything.



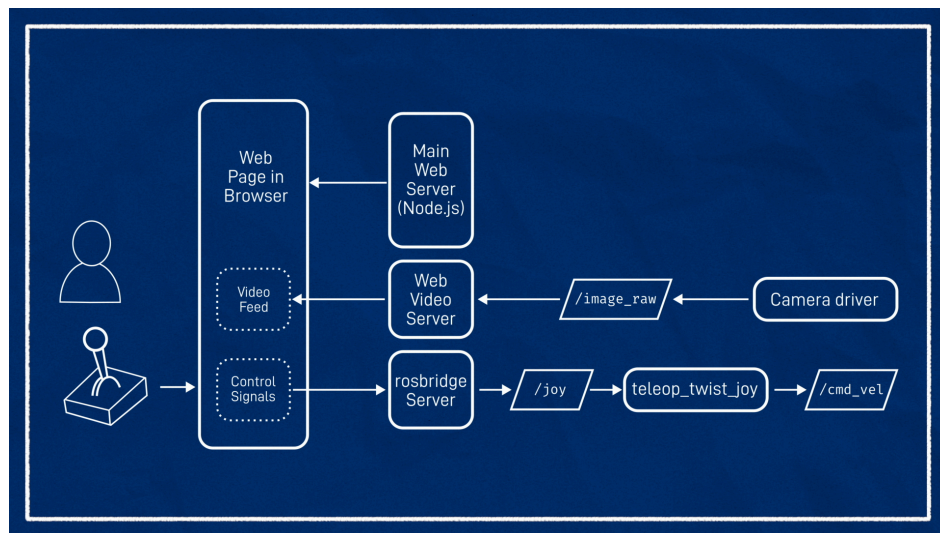
I think that Valve's [Steam Deck](#) and similar products are going to be really valuable for this kind of development in the future but unfortunately right now availability is limited (and it has not even been released here in Australia).

We're going to have a go at doing this ourselves. The following method (and the one in the subsequent section) runs in a browser, which means that we are not limited to watching from a computer, and can instead use phones and tablets as well. For control, we can connect USB or Bluetooth controllers like the one below (that is the one I used, though I would not necessarily recommend it), or on-screen buttons.

The instructions will be kept fairly brief, but are hopefully sufficient to guide through the process.



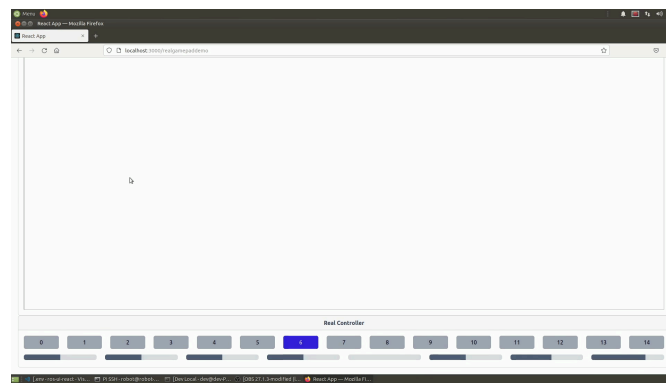
Warning, this next procedure is something that I hacked together almost two years ago and have not sunk any time into improving since. It is not an example of good code, and it does not run very well, it is just an interesting proof of concept.



Set up web server

The first step is to get a web server running. We'll serve it from the dev machine, and also test it on there.

- Install Node.js v16 on the dev machine [from NodeSource binary distributions](#)
- Clone my [ros-ui-react](#) package somewhere on the machine (note that this is not a ROS package)
- In the root of `ros-ui-react`, run `npm install` to download dependencies and `npm run build` to build/rebuild (if you make future modifications, you will need to rerun `npm run build`).
- Using a text editor, make the following changes to files in the `example/` subdirectory:
 - Create a file called `.env` (i.e. `example/.env`) with the contents `SKIP_PREFLIGHT_CHECK=true`.
 - (Alternatively you could run `echo SKIP_PREFLIGHT_CHECK=true > .env` from a terminal in the example directory)
 - In `src/RealGamepadDemo.js` and `src/SimGamepadDemo.js`, change the IP addresses (and if necessary, ports and ROS topics) for the `ImageStream` and `RealGamepad` elements. More on this below.
- Run `npm install` from the example directory to install dependencies
- Run `npm start` from the example directory to start the web server
- A browser should automatically pop up once the page is built. If it doesn't, navigate to `localhost:3000/`. Try clicking on "Real Gamepad Demo", and if you have a gamepad connected, pressing buttons on it should cause them to appear on screen.



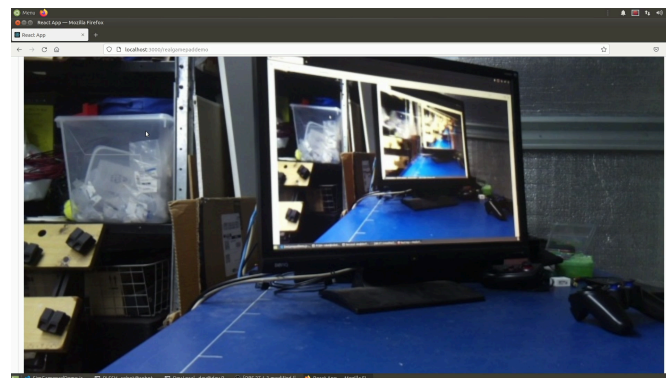
Set up image feed

Something immediately obvious on this page will be the image display that is broken. To fix this, we will run a small server directly on the Pi that serves up Image topics as webcam-style image feeds.

On the Pi (can alternatively be done on the dev machine):

- Install `ros-foxy-async-web-server-cpp` (with apt)
- Clone the ROS2 branch of `web_video_server` into a ROS workspace (e.g. our `robot_ws`) with `git clone -b ros2 [https://github.com/RobotWebTools/web_video_server.git](https://github.com/RobotWebTools/web_video_server.git)` (`[https://github.com/RobotWebTools/web_video_server.git](https://github.com/RobotWebTools/web_video_server.git)`)
- Rebuild the workspace with colcon
- In two different tabs, source the workspace, launch the camera driver (like normal), and run `ros2 run web_video_server web_video_server`

Then, back in the browser on the dev machine the image feed should be working. If not, try navigating to `<pi IP address>:8080`. This should list the available topics and you can click on them to check they work. If all is fine, check the IP address and ROS topic set in `ros-ui-react/example/src/RealGamepadDemo.js`.



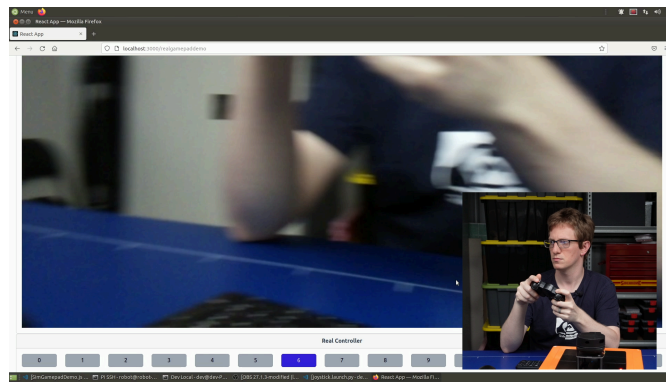
Set up controls

Now that the image feed is coming through, we need the control signals to be published to ROS on `/joy`. We'll do this using `rosbridge`, a suite of libraries and tools that (among other things) let us interact with ROS in the browser with Javascript, communicating to a server via WebSockets.

- Install `rosbridge_suite` on whichever device will be receiving control commands with `joystick.launch.py` (in my case it was the dev machine)
 - Normally this should work with `sudo apt install ros-foxy-rosbridge-suite`
 - However I had some trouble with that and had to [build it from source](#)
- Run `ros2 run rosbridge_server rosbridge_websocket`
- Modify `joystick.launch.py` to not launch `joy_node` (since the `/joy` topic will now come from rosbridge). You may want to add launch arguments to toggle this behaviour.
- Launch `joystick.launch.py`
- Start your `ros2_control` controllers on the robot (ensure your robot is in a safe position)
- Open the browser on the dev machine and start driving your robot around

If the controls are not working correctly...

- Visually check that the buttons are showing up on the screen, and that your enable buttons are the same as they were for a direct connection (and if not, modify `joystick.yaml`)
- Echo the `/joy` topic to ensure that the button presses are coming through



Run on phone/tablet

Now that we have all the desired functionality in the browser, we can try it on other devices.

- Connect your device to the same Wi-Fi network
- Connect a gamepad to your device via USB or Bluetooth (optional, you can instead use the simulated gamepad with on-screen buttons)
- Navigate to the address in your browser
- Optionally "Add the webpage to your home screen" (easier to access and removes address bar etc taking up space).
- Note that in all likelihood, the button configuration will be different so you may need to adjust `joystick.yaml`
- If using the on-screen buttons, you will probably want to set `require_enable_button` to false (in `joystick.yaml`) as using the dead-man's switch does not work well on the touchscreen

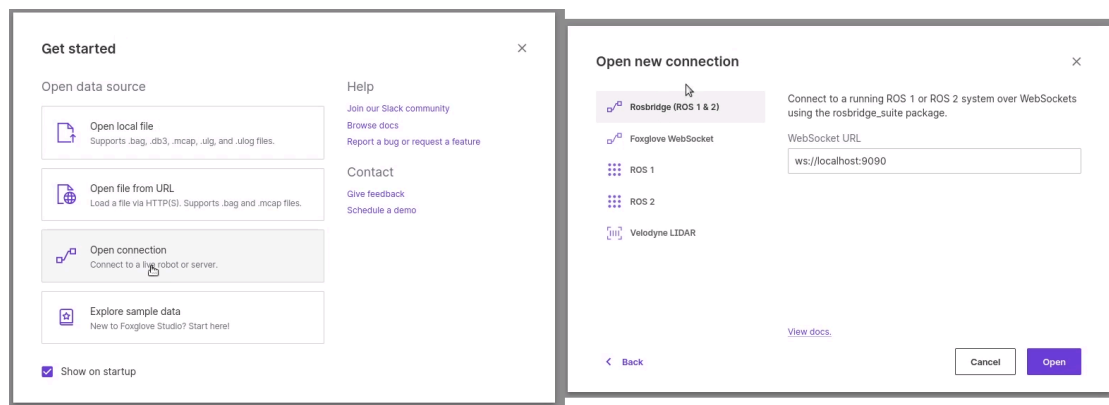


Foxglove

Shortly after I wrote that tool at the start of 2021, a group called Foxglove sprung up with a new (or revamped) tool called Foxglove Studio, and since then it's gotten better and better. It is also browser-based, and utilises `rosbridge_server`. However it is missing one nice feature, as we'll see at the end of this section.

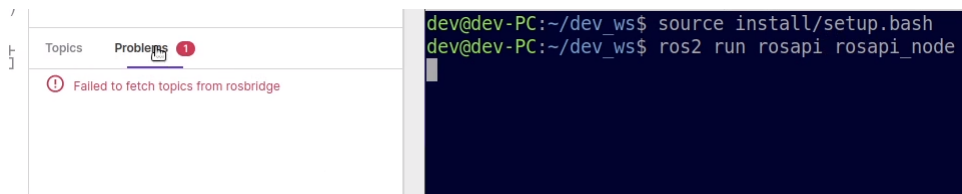


To start Foxglove, you can either install the application on your computer, or navigate to <https://studio.foxglove.dev/>. It will prompt to select a data source (in this case we want to connect to a live robot, which includes Gazebo), and then to select a connection type. It may be tempting to select "ROS 2", however this mode is very limited (and only works in the app). Instead, select the default option (Rosbridge).



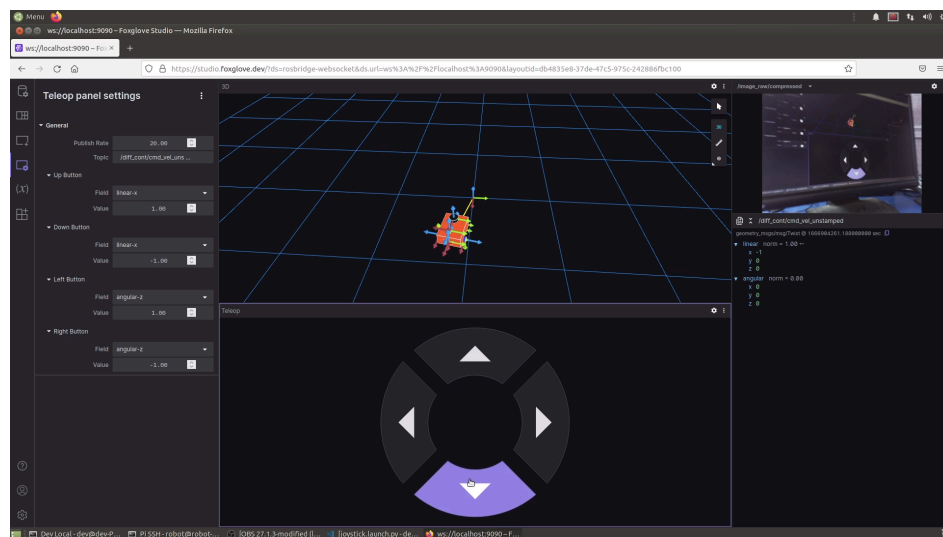
Things won't be working quite yet, and we'll find that it is failing to fetch the topic list. This is because we also need to run `rosapi_node`, which is part of the `rosapi` package and will be installed as part of the same process as `rosbridge_server` (`rosbridge_suite`).

Note: to make things easier, instead of running `rosbridge_server` and `rosapi_node` separately, you can run `ros2 launch rosbridge_server rosbridge_websocket_launch.xml` to launch them together!



After running the `rosapi_node`, everything should be up and going! Explore the interface, adding different panels, and see what you can do.

Note: If you try to display lidar data in the Foxglove 3D panel, it may not appear to be working. Try adjusting the display settings for it, particularly the size, colour type, and opacity, and it should become clearly visible.



One useful panel is the Teleop panel, from which we can drive our robot. This panel directly publishes `Twist` messages (the topic must be configured in the panel settings). Unfortunately, there is currently no way to connect a gamepad directly to Foxglove, and use the bluetooth controller like we did previously. Hopefully this will be added one day, and in the meantime I have been working on an [example extension panel](#) that you can install to try it out.

Extra Tips

Below are a bunch of extra tips, tricks, and improvements for our robot's teleop system.

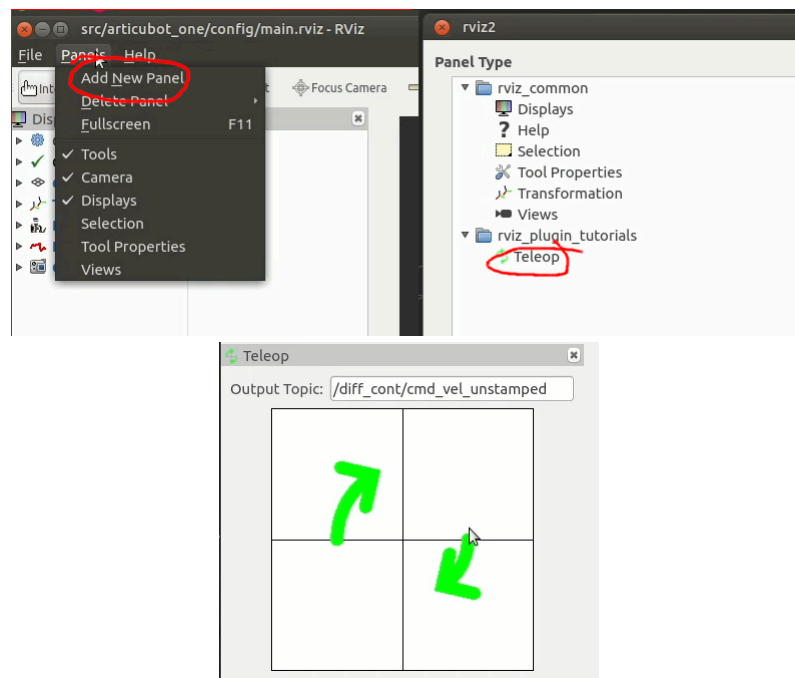
Teleop from RViz

Sometimes it would be convenient to be able to run teleop directly from RViz. The ROS 1 RViz plugin tutorials had an example that did exactly this, unfortunately, the port for ROS 2 was never properly completed. While there is no apt package available, there is a branch on the git repo that works, so we can clone it to our workspace and build it.

```
git clone -b port-plugin-tutorials [https://github.com/ros-visualization/visualization_tutorials.git](https://github.com/ros-visualization/visualization_tutorials.git)
```

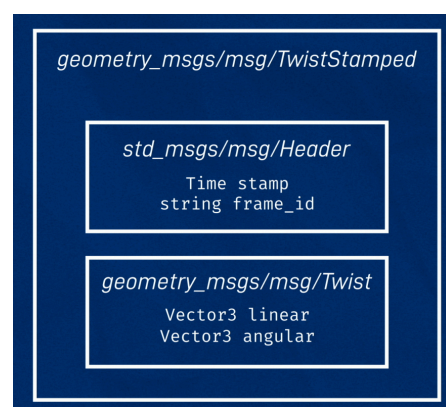
Then, next time we run RViz, we need to make sure we **source our workspace** first - this will tell RViz how to find the plugin.

Once RViz is up and running we can add a teleop panel, set the output message, and drive it around! Remember to stop your joystick launch first if it's still running.



Stamped/Unstamped Twist

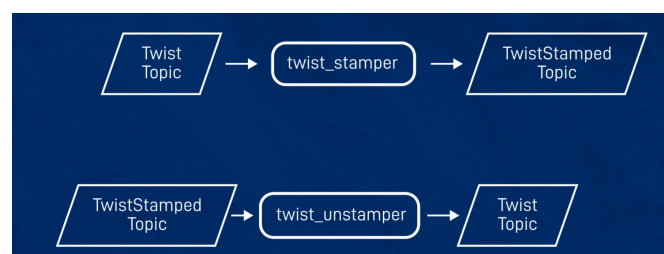
Something you might have noticed keeps coming up is `Twist` and `TwistStamped`. A `TwistStamped` is basically just a `Twist` with a time and transform frame associated with it.



Why would we want that?

Well, let's say we have network troubles and suddenly our velocity requests are taking 10 seconds to get through. Executing commands we sent in the past can potentially be very dangerous. By checking the time associated with the command, the controller could reject any requests that are too old, keeping things safe. Also, by using the provided reference frame, a controller could transform the request appropriately.

When I started this project a couple of years ago, the `diff_drive_controller` only accepted `TwistStamped` - quite reasonably. But all the teleop tools published `Twist` instead. So I wrote a tool called `twist_stamper` that simply takes a twist in and spits out one that is stamped with the current time. It also comes with another node called `twist_unstamper` that does the opposite. It may be that you've configured the controller to take an unstamped velocity, but then you have a path planner that spits out twiststamped. You can just strip the timestamp off and move on.



Two things to keep in mind when doing this are:

- Using `twist_stamper` doesn't magically make things safe, there are still opportunities for delays to be introduced (especially if you were to run the stamper on the robot itself with the gamepad on another computer). But it's one step better.
- If your `TwistStamped` source computer (dev machine in this case) and the Pi don't have clocks synced, you'll have problems, but you were probably going to have problems with SLAM and other similar tasks anyway.

A very brief summary of how to upgrade our project to use `twist_stamper`:

- Install `twist_stamper` (`sudo apt install ros-foxy-twist-stamper`)
- Add another node to our `joystick.launch.py`, that runs `twist_stamper`
- Configure the remapping, `twist_stamper` expects you to remap `/cmd_vel_in` and `/cmd_vel_out`.

- To create a drop-in replacement, we want to remap from `/cmd_vel_in` to `/diff_cont/cmd_vel_unstamped` and from `/cmd_vel_out` to `/diff_cont/cmd_vel`
- Alternatively, we could remove our current remapping in `teleop_twist_joy`, so that it defaults to `/cmd_vel`, and then simply remap `/cmd_vel_in` to `/cmd_vel`.
- Update `my_controller_params.yaml` to set `use_stamped_vel` to `true`

```
twist_stamper = Node(
    package='twist_stamper',
    executable='twist_stamper',
    remappings=[('/cmd_vel_in', '/diff_cont/cmd_vel_unstamped'),
                ('/cmd_vel_out', '/diff_cont/cmd_vel')]
)
```

Then start everything back up and it should work! The procedure to use `twist_unstamper` is very similar.

Note that for the rest of the tutorials I will be assuming that this has NOT been done, and that we are still using unstamped command velocities.

Gazebo/Sim Time

As always, we should account for sim mode when using Gazebo.

We can simply add a launch argument called `use_sim_time`, defaulted to `false`, and map that as an extra parameter to each of these nodes (this process is the same as previous tutorials).

```
joy_node = Node(
    package='joy',
    executable='joy_node',
    ### -----
    parameters=[joy_params, {'use_sim_time': use_sim_time}]
    ### ^^ Updated Line ^^
)
```

Now when we are going to use Gazebo, we should launch `joystick.launch.py` with `use_sim_time=true`.

Launch files

On the topic of launching, we can take things a step further by including `joystick.launch.py` in our other launch files. As an example, I chose to include mine in `launch_sim.launch.py` with `use_sim_time` set to `true` so that whenever I am running a simulation I have the controls available (as always remember to add it into the `LaunchDescription` at the end too). If this becomes an issue we can easily disable it later.

```
joystick = IncludeLaunchDescription(
    PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory(package_name), 'launch', 'joystick.launch.py'
    )]), launch_arguments={'use_sim_time': 'true'}.items()
)
```

If you're connecting the gamepad directly to the Pi when driving it, you could also add it to the `launch_robot.launch.py` (remember to leave `use_sim_time` off or false).

An even better approach would be to add extra arguments to these launch files that enable or disable the joystick inclusion for easy toggling!

Using the old Linux Joystick Interface

Something mentioned in the previous tutorial is that Linux has two different drivers for interfacing with joysticks, the older `joydev` interface and the newer `evdev` interface. The new way is usually preferred but sometimes the old way may be required (especially with unusual configurations, like Docker).

If you're having trouble getting your gamepad to work with ROS, but it's showing up fine in `jstest` or `jstest-gtk`, then try the old driver. The package is `joy_linux` and the node is `joy_linux_node`. Most of the other parameters are the same, except we set the device as a Linux device path, e.g. `device:=/dev/input/js0`.

New teleop_tools package

One last thing to briefly mention (in case you stumble across it) is that there is actually another package you can use for the joy-to-twist conversion and that's `joy_teleop` from the `teleop_tools` stack. I've not actually used it myself, but it seems to be a much more powerful tool that lets you map whatever buttons and axes to any messages and services that you want, so that could be `Twist` or `TwistStamped` but it could also trigger actions or send joint references for an arm or something.

For the rest of these tutorials, however, we'll stick with `teleop_twist_joy`.

Conclusion

Together these last two tutorials have been a fairly comprehensive overview of teleoperation of our robot. Now that teleop works, the next step is automation, and we're going to start looking at some of the more complex algorithms we can experiment with. Specifically, the next few tutorials will be on:

- Running SLAM with our lidar using `slam_toolbox`
- Autonomous navigation with Nav2
- Using the camera as an input for our robot to chase a moving target

**Markus_Messerschmidt**

Dec '22

Hello.
 which ip addresses are taken? first line that of the robot and in the second line that of the dev machine?
 Unfortunately, the npm server is not running as desired for me.
 Do I have to rebuild the root directory if there are changes or just run npm install and then npm start in the example folder?

[1 reply](#)**JoshNewans**

Jan '23

Markus_Messerschmidt:

which ip addresses are taken? first line that of the robot and in the second line that of the dev machine?

Are you referring to [here](#) where I set the IP addresses in the web page?

The IP address for the ImageStream should be whichever is running web_video_server and the one for RealGamepad/SimGamepad should be whichever is running rosbriidge. In my example the former was the Pi and the latter was the dev machine. Make sure you do it for both of the pages (sim and real).

Markus_Messerschmidt:

Unfortunately, the npm server is not running as desired for me.

In what way?

Markus_Messerschmidt:

Do I have to rebuild the root directory if there are changes or just run npm install and then npm start in the example folder?

Yeah, good question, I get very confused by it all. In theory when you npm start it should auto pick up changes. I think in this case because there are actually two levels, the library and the example page, it doesn't seem to work properly. And to make things confusing the "conceptual top level" (the example page) code is nested *inside* the lower level (the library).

So if you are only tweaking something at the example level you should not need to rebuild, but if you tweak the library code I think you need to rebuild it then rerun the example/page/server bit.

This is an area I am *very* weak in so I probably did it all wrong, and I also wrote it a couple of years ago.

I am (slowly) working on a Foxglove extension to totally replace this which will be a far better approach!

**Sasffm**

Feb '23

Hi there,
 i am having a strange behaviour.
 While using a gamepad my robot turns right when i am steering to the left and it turns left when i am steering to the right.
 Where is my mistake here?
 Thanks

[1 reply](#)**JoshNewans**[▶ Sasffm](#) Feb '23

Hey!
 The simplest answer is probably: in the teleop_twist_joy params, try just flipping the sign of the angular Z value.
 You can also step through and echo /joy and your cmd_vel topic to check what the values are at each stage.

[2 replies](#)**Sasffm**[▶ JoshNewans](#) Feb '23

Thanks for the fast answer.
 Meanwhile i have found the problem.
 I had to switch the motor cables on the controller.

**John_Nam**

Mar '23

Thank you Josh your great video.
 my robot turn left with joystick but my rviz turn right...
 in your tutorial... you explained I think.
 but I couldn't find your explain...
 where should I fix it...
 Anyone knows this issue and how to fix it please let me know

Regards,

John...

[1 reply](#)**John_Nam**

Mar '23

JoshNewans:

an also step through and echo /

fixed with hardware connection ... motor connection with the arduino.

**John_Nam**

Mar '23

interrupt each other → camera.launch.py and launch_robot.launch.py
 in ros2_control.xacro

/dev/serial/by-path/platform-fd ~~~~ tried avoid interrupt arduino and rplidar
 rplidar.launch.py and launch_robot.launch.py is ok

issue: during using rviz2 -d ~~~... can not use camera...

[1 reply](#)**JoshNewans**[▶ John_Nam](#) Mar '23

Hey John, sorry I'm a little confused by your replies - can you please clarify what the current problems are?

- The first one seemed to be an issue with it showing the turns the wrong direction in RViz, but then possibly you got that fixed?
- The second I'm not sure if you are saying there is a clash between the camera and the lidar? Or the Arduino?

If the camera is not showing up in RViz, make sure you are using an Image display (not a Camera as it can be trickier to get going) and check with the dropdown that the topic is actually being published. There was an issue I didn't explain clearly at this point that the simulated camera and real camera were set up to publish on slightly different topics.

If you can explain the issue more clearly I'm happy to try and help 😊

1 reply



John_Nam

JoshNewans Mar '23

There is a clash between the camera and the Arduino.
the lidar and the Arduino(robot) looks fine at the moment.

in the ros2_control.xacro

```
diffdrive_arduino/DiffDriveArduino
left_wheel_joint
right_wheel_joint
30
```

```
<param name="device">/dev/ttyUSB1</param>
```

```
**usb0 is lidar, usb1 is robot
---o-----
```

```
/dev/serial/by-path/platform-fd500000.pcie-pci-0000:01:00.0-usb-0:1.3.1:1.0-port0
```

both I tested but same issue...

```
<param name="baud_rate">57600</param>
<param name="timeout">1000</param>
<param name="enc_counts_per_rev">3436</param>
</hardware>
```

in the pi

```
mate_bot@pi:~$ ls /dev/ttyUSB*
/dev/ttyUSB0 /dev/ttyUSB1
```

```
mate_bot@pi:~$ ls /dev/serial/by-path
platform-fd500000.pcie-pci-0000:01:00.0-usb-0:1.3.1:1.0-port0
platform-fd500000.pcie-pci-0000:01:00.0-usb-0:1.3.3:1.0-port0
```

in the rviz2

reset button

robot and lidar working

camera warn... camera info: No camera info received on [/camera/camera_info] Topic may not exist

if only run camera launch file, rqt_image_view is working.

Please let me know how to fix the crash with camera and robot(arduino)
I searched google it but couldn't find the solution
sorry interrupt and my poor English

1 reply



John_Nam

Mar '23

camera issue with web server.
any one have an idea please let me know

<http://localhost:3000/realgamepaddemo>

There is no video stream.

I check the topic list at the dev machine.

There is a /image_raw as below.

```
dev-pc@Dev-PC:~/dev_ws$ ros2 topic list
/camera_info
/diff_cont/cmd_vel_unstamped
/image_raw
/image_raw/compressed
/image_raw/compressedDepth
/image_raw/theora
/joy
/joy/set_feedback
/parameter_events
/rosout
```

Tested with the rqt_image_view is working at the dev machine
dev-pc@Dev-PC:~/dev_ws\$ ros2 run rqt_image_view rqt_image_view

The pi side web_video_server is working well I think
mate_bot@pi:~/robot_ws\$ ros2 run web_video_server web_video_server
[INFO] [1679537318.652438037] [web_video_server]: Waiting For connections on 0.0.0.0:8080

1 reply



JoshNewans

Mar '23

Hey John, ok let's unpack a few of these things.

1 John_Nam:

There is a clash between the camera and the Arduino.

Unless you have a camera that works over serial, it shouldn't be possible for these to clash.

1 John_Nam:

in the rviz2

reset button

robot and lidar working

camera warn... camera info: No camera info received on [/camera/camera_info] Topic may not exist

if only run camera launch file, rqt_image_view is working.

Ah I think this might be a problem that I've been meaning to look into. In RViz there are two ways to look at camera data - an "Image" display and a "Camera" display. The Camera display expects there to be not just an image topic, but a CameraInfo topic to go along with it. When we simulate with Gazebo it creates both topics, but I think the camera driver I'm using is NOT creating the CameraInfo topic. I think this might be because it doesn't have calibration data but I'm not 100% sure.

Everything should work fine regardless, as all the other tutorials rely only on the image data not the camera info. And in RViz you can just change it to an "Image" display.

1 John_Nam:

(post deleted by author)

I saw you've deleted this one, but to answer the question anyway in case someone else finds this: if you are running it with the "real controller" it won't show up the buttons until it has seen the gamepad. Usually that will happen the first time you press a button after loading the page.

1 John_Nam:

camera issue with web server.

any one have an idea please let me know

<http://localhost:3000/realgamepaddemo>
There is no video stream.
I check the topic list at the dev machine.

At a guess, I'd say you're not running the web video server which needs to also be running and it converts the image topics to a web-friendly format. [Here](#) is the repo for it, I do show that in the "Part B" video.

As a final note, I am still working on my plugin for Foxglove that makes that whole janky web demo completely redundant, so stay tuned for that 🤖

Hope that helps, please reply if you have further troubles.

1 reply



John_Nam

Mar '23

for the rviz2
git clone issue...

git clone -b port_plugin-tutorials [GitHub - ros-visualization/visualization_tutorials: Tutorials related to using and extending RViz and interactive markers.](#)

cloning into 'visualization_tutorials'...
fatal: Remote branch port-plugin-tutorials not found in upstream origin

Thank you very much your wonderful lecture
I am not an IT person... so sometime very easy things to you... very difficult to me...
your 20 min lecturing... to me more than 1 week learning...
Why so many stupid questions and issues



MarcW

► JoshNewans Mar '23

Hey Josh,
I use a GStreamer node that I found on github: [GitHub - clydemcqueen/gscam2: ROS2 camera driver for GStreamer-based video streams, supports intra-process communication](#)
It's a ROS2 port, publishes the camera info and worked fine for me. The Jetson Nano has some issues with MIPI cameras which is why I went this way.

1 reply



RobotRoss

► MarcW Apr '23

Teleop 14 Part A. Thank you for this excellent tutorial which brings together essential info on understanding teleoperation. And the joy_tester package is very helpful in determining the Button and Axis numbering in a ROS2 system-not clearly explained anywhere else as clearly, and pleased that it now can do a binary instal on a Ubuntu 22.04 machine, ros-humble-joy-tester.



Rakshit_Soni

May '23

Hey Josh,
I am trying to subscribe the Joy message from joy_linux node to a custom node. Basically I want to convert the Joy message to Float64 for speed(linear) and float64(wheel angle) for a robot behaving like Ackermann steered. I have written the pub/sub nodes for these two messages but getting error in the code. Also, for Ackermann steered robot, how I can control the robot. As there is no support for Ackermann steered robot in ROS2, no ros2-control is there, no gazebo plugin except for toyota prius example but that doesn't work. I am new to ROS so need some advice in these issues.
If you can help me with this, it will be great. Using ROS2 Humble, Ubuntu 22.04

THE CODE:

```
#!/usr/bin/env python
import sys
import rclpy
import pdb
from sensor_msgs.msg import Joy
from std_msgs.msg import Float64
import subprocess, time
from rclpy.node import Node
from std_msgs.msg import String

def rescale(x, oldmin, oldmax, newmin, newmax):
    r = (x-oldmin)/(oldmax-oldmin)
    out = newmin + r*(newmax-newmin)
    return out

class Joystick2speed(Node):

    def __init__(self):
        super().__init__('joystick2speed')
        self.pub = self.create_publisher(Float64, 'speedcmd_metersec', 1)
        self.sub = self.create_subscription('/joy', Joy, self.joy_callback, 1)
        self.buffer = []
        for i in range(0,100):
            self.buffer.append('X')
        self.buffer_index = 0
        self.get_logger().info("Speed node has been initialized")
    def joy_callback(self, data):
        joystick_y = -data.y
        deadmanzone_min = -0.2
        deadmanzone_max = 0.2
        max_fwd_speed_per_second = 2
        max_bkwd_speed_per_sec = 2
        if joystick_y > deadmanzone_min and joystick_y < deadmanzone_max:
            velocity = 0
        elif joystick_y > 0:
            velocity = rescale(joystick_y, deadmanzone_max, 1., 0., max_fwd_speed_per_second)
        elif joystick_y < 0:
            velocity = rescale(joystick_y, -1., deadmanzone_min, -max_bkwd_speed_per_sec, 0.)
        msg_out = Float64(velocity)
        print(velocity)
        self.pub.publish(msg_out)

def main(args = None):
    rclpy.init(args=args)
    joystick2speed = Joystick2speed()
    rclpy.spin(joystick2speed)
    rclpy.shutdown()
    if __name__ == '__main__':
        main()
```

Error: Traceback (most recent call last):
File "/home/computing/podcar_2/src/pod2_bringup/scripts/joystick2wheelAngle.py", line 34, in main()
File "/home/computing/podcar_2/src/pod2_bringup/scripts/joystick2wheelAngle.py", line 30, in main
joystick2wheel = Joystick_2_wheel()
File "/home/computing/podcar_2/src/pod2_bringup/scripts/joystick2wheelAngle.py", line 16, in init
self.pub = self.create_publisher('wheelAngleCmd', Float64, 10)

```
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rcipy/node.py", line 1273, in create_publisher
final_topic = self.resolve_topic_name(topic)
File "/opt/ros/humble/local/lib/python3.10/dist-packages/rcipy/node.py", line 1226, in resolve_topic_name
return _rcipy.rcipy_resolve_name(self.handle, topic, only_expand, False)
TypeError: rcipy_resolve_name(): incompatible function arguments. The following argument types are supported:
1. (arg0: rcipy::Node, arg1: str, arg2: bool, arg3: bool) -> str

Invoked with: <rcipy._rcipy_pybind11.Node object at 0x7ffb31f413b0>, <class 'std_msgs.msg._float64.Float64'>, False, False
```

S

Stardobas

Nov '23

Hey Josh,

I've destroyed all system trying to make "control your robot from your phone"... 😞 I'm not able to install nothing (apt install) anymore, this give me a strange irreversible error in dependencies...(navigation need cereslib that can't be installed for example...).

Now, first I try again in new system, need to know if in your method (node.js...stream camera...game pad controller) **is possible to stream Rviz too...?**... I would be have camera, controller, and Rviz working...

PS, in my system I don't use internet but only local net, and now I'm trying to work all system in raspberry, using navigation and mapping too...is very slow but I think in future the pi5 help more to do this. Following your fantastic tutorial in my pc (simulated robot) all work except remote control from web because I'm afraid about trying tutorial to destroy pc system too like raspberry).

Thank you

F

Foolsim_simulate

Apr '24

currently, i can run ros2 run rosbridge_server rosbridge_websocket on workspace ros-ui-react/example is succesfull , so I need to install git clone [GitHub - RobotWebTools/rosbridge_suite: Server Implementations of the rosbridge v2 Protocol](#) on workspace : Dev_ws or not ? because i tried to use a joystick. Launch is a real robot but that does not move (I have adjusted joystick.launch.py)

R

RawStone

Aug '24

Hey Josh! Thank you so much for the lecture series.

I am trying to run the articubot_one repo on my own system and hardware. I've made all the proper motor connections and have gotten all the launch-scripts and nodes to work.

I've run into a weird error and I will try my best to describe it so bear with me.

When I try to operate the robot via Teleop commands, the robot only takes the command once every few seconds. It moves once every few seconds and stops after its 'set' timeout period(which according to my code is 2secs). i.e. if it doesn't receive the next commands with in 2 seconds of the prev command, the motor stops moving. Now i call this behaviour random because sometimes the motor receives the commands 2 seconds apart from each other, sometimes they're 5 seconds apart, sometimes even 30 secs apart. It does not move in those brief periods of time.

- I have tried running the robot via miniterm and I continuously kept pasting the command "o 255 255". The motor constantly keeps moving for however long i keep pasting the command which leads me to believe there is nothing wrong in the circuitry corresponding to Arduino.
- I have checked my keyboard and made sure that there is nothing wrong with the keys & keystroke signals are continually sent.

Can you suggest what else I can do or if anyone has any idea what could be wrong, help would be greatly appreciated.

Thanks in advance

1 reply

R

RawStone

RawStone

Oct '24

So I found the issue.

The problem appears to be with my motor of choice. well the encoder on the motor is whats causing the problem to be precise. Instead of using the provided 6pin female header, i bypassed it and gave the connections directly to the motors on their terminals on the bottom and worked like a charm. I'll have to figure out what exactly is causing this problem on the encoder.

Leaving this here in case anyone in the future has a similar issue.

Cheers!

[Continue Discussion](#)



